

3 BOXES LUXURY

Technical Configuration Documentation

Complete Step-by-Step Guide for Vercel Deployment,
Database Configuration, and Shopify Integration

Document	Details
Project	3 BOXES LUXURY - Luxury Gifting E-Commerce Platform
Framework	Next.js 16 with App Router (TypeScript)
Deployment	Vercel (Serverless)
Live URL	https://my-project-sepia-seven-42.vercel.app
Database	PostgreSQL (Neon) + SQLite (Local Dev)
Commerce	Shopify Headless Storefront
Version	1.0.0
Date	March 2025

CONFIDENTIAL - This document contains sensitive credentials and API keys. Handle with care.

Table of Contents

1. Project Overview	4
1.1 Technology Stack	4
1.2 Application Architecture	4
1.3 Current Deployment Status	5
2. Vercel Deployment - Step by Step	5
2.1 Prerequisites	5
2.2 Vercel Account Setup	6
2.3 Project Deployment	6
2.4 Vercel Build Configuration	6
2.5 Build Script Details	7
3. Environment Variables Configuration	7
3.1 Required Environment Variables	7
3.2 Setting Environment Variables on Vercel	7
3.3 Variable Details	8
4. Neon PostgreSQL Database Setup	9
4.1 Creating a Neon Database	9
4.2 Neon Free Tier Limits	9
4.3 Connecting Neon to Vercel	10
4.4 Verifying Database Connection	11
5. Shopify Integration Configuration	11
5.1 Shopify Store Details	11
5.2 Creating Shopify API Credentials	11
5.3 Shopify API Fallback System	12
6. Database Schema (Prisma)	13
6.1 Core E-Commerce Models	13
6.2 User and Authentication Models	13
6.3 Corporate Gifting Models	13

6.4 Admin and Operations Models	14
7. API Routes Reference	14
7.1 Authentication Routes	14
7.2 Product and Category Routes	15
7.3 Shopping and Checkout Routes	15
7.4 Admin Routes	16
7.5 Corporate Gifting Routes	16
8. Complete Credentials Reference	17
8.1 Vercel Deployment	17
8.2 Shopify Store	17
8.3 Database	18
9. Troubleshooting Guide	18
9.1 Build Failures	18
9.2 Runtime Errors	18
9.3 Quick Diagnostic Commands	19
10. Complete Setup Checklist	19

1. Project Overview

3 BOXES LUXURY is a premium luxury gifting e-commerce platform built as a headless Shopify storefront. The application provides a complete shopping experience with product browsing, cart management, checkout, order tracking, corporate gifting portal, admin dashboard, AI-powered gift recommendations, multi-currency support, and internationalization (i18n) for 9 languages.

1.1 Technology Stack

Layer	Technology	Version
Framework	Next.js (App Router)	16.1.1
Language	TypeScript	5.x
Styling	Tailwind CSS + shadcn/ui	4.x
ORM	Prisma	6.11.1
State (Client)	Zustand	5.0.6
State (Server)	TanStack React Query	5.82.0
Auth	NextAuth.js (Custom JWT)	4.24.11
Animations	Framer Motion	12.23.2
Charts	Recharts	2.15.4
Database (Prod)	PostgreSQL (Neon)	-
Database (Dev)	SQLite	-
Commerce	Shopify Admin API + Storefront API	2024-10
Deployment	Vercel (Serverless Functions)	-
Package Manager	Bun	-

1.2 Application Architecture

The application follows a headless commerce architecture where Shopify serves as the product and inventory backend, while the Next.js application handles the storefront UI, user authentication, cart management, order processing, and corporate gifting workflows. A Shopify Admin API fallback layer ensures products and categories are always available even when the PostgreSQL database is unreachable.

Component	Role
Next.js Frontend	Server-side rendered UI with client-side hydration

API Routes (/api/*)	RESTful backend endpoints for all business logic
Prisma ORM	Database abstraction layer (SQLite local / PostgreSQL prod)
Shopify Admin API	Primary product/category data source with fallback caching
Shopify Storefront API	Checkout and cart integration with Shopify
Vercel Serverless	Deployment platform with edge functions and CDN
Neon PostgreSQL	Serverless PostgreSQL for production data persistence

1.3 Current Deployment Status

Feature	Status	Notes
Frontend UI	WORKING	All pages render correctly
Products (57 items)	WORKING	Via Shopify Admin API fallback
Categories (15 items)	WORKING	Derived from Shopify collections
Shopping Cart	NEEDS DB	Requires PostgreSQL connection
Orders	NEEDS DB	Requires PostgreSQL connection
User Authentication	NEEDS DB	Requires PostgreSQL connection
Admin Dashboard	NEEDS DB	Requires PostgreSQL connection
Corporate Gifting	NEEDS DB	Requires PostgreSQL connection
AI Recommendations	NEEDS DB	Requires PostgreSQL connection
Multi-Currency	PARTIAL	Config exists, needs DB for rates
i18n (9 languages)	WORKING	Translation files embedded

2. Vercel Deployment - Step by Step

2.1 Prerequisites

- Node.js 18+ or Bun runtime installed
- A Vercel account (free tier works)
- Vercel CLI installed: `npm i -g vercel`
- A Neon PostgreSQL database (free tier works)
- Shopify Partner account with store access
- Git repository with the project code

2.2 Vercel Account Setup

Step 1: Go to <https://vercel.com> and sign up or log in

Step 2: Connect your Git provider (GitHub/GitLab/Bitbucket)

Step 3: Install the Vercel CLI globally:

```
npm install -g vercel
```

Step 4: Login to Vercel from CLI:

```
vercel login
```

Step 5: Verify login:

```
vercel whoami
```

2.3 Project Deployment

Step 1: Navigate to the project directory:

```
cd /home/z/my-project
```

Step 2: Link the project to Vercel (first time only):

```
vercel link
```

When prompted, select your team/scope and accept the default project name or provide a custom one.

Step 3: Deploy to preview:

```
vercel
```

Step 4: Deploy to production:

```
vercel --prod
```

Note: The vercel-build script in package.json automatically switches Prisma from SQLite to PostgreSQL and runs prisma db push before building.

2.4 Vercel Build Configuration

The project uses a custom vercel.json file that controls the build process:

Setting	Value	Purpose
buildCommand	sed + prisma generate + prisma db push + next build	Switches SQLite to PostgreSQL, generates Prisma client, pushes schema, then builds
installCommand	bun install	Uses Bun for faster installs
framework	nextjs	Auto-detected Next.js framework
regions	bom1 (Mumbai)	Deployed to Mumbai region for India-based audience

2.5 Build Script Details

The vercel-build script in package.json performs the following operations in sequence:

Step	Command	Description
1	sed -i 's/provider = "sqlite"/ provider = "postgresql"/g' prisma/schema.prisma	Switches Prisma datasource from SQLite to PostgreSQL for production
2	prisma generate	Generates Prisma Client with PostgreSQL provider
3	prisma db push --accept-data-loss	Pushes schema to Neon PostgreSQL and creates tables
4	next build	Builds the Next.js application for production

Note: The --accept-data-loss flag is used because schema changes in development may not be backward-compatible. In production, use prisma migrate for safer migrations.

3. Environment Variables Configuration

Environment variables are the most critical part of the deployment. They must be configured correctly on Vercel for the application to function properly. Below is the complete list of required environment variables with their current values and descriptions.

3.1 Required Environment Variables

Variable	Value	Required
DATABASE_URL	postgresql://user:pass@ep-xxx. neon.tech/dbname?sslmode=require	YES
SHOPIFY_STORE_DOMAIN	3boxesluxury-2.myshopify.com	YES
SHOPIFY_STOREFRONT_ACCESS_TO KEN	09124abbd07ca423DB9488B3DC2C737F	YES
SHOPIFY_ADMIN_API_TOKEN	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	YES
NEXTAUTH_SECRET	(auto-generated or custom)	YES
NEXTAUTH_URL	https://my-project-sepia-seven-42.vercel.app	YES

3.2 Setting Environment Variables on Vercel

Step 1: Go to your Vercel Dashboard: <https://vercel.com/dashboard>

Step 2: Select your project (3 BOXES LUXURY)

Step 3: Navigate to Settings > Environment Variables

Step 4: Add each variable one by one:

For each variable, enter the Name and Value, then select the environments (Production, Preview, Development) where it should be available.

Step 5: Alternatively, use the Vercel CLI to set environment variables:

```
vercel env add DATABASE_URL production
vercel env add SHOPIFY_STORE_DOMAIN production
vercel env add SHOPIFY_ADMIN_API_TOKEN production
vercel env add SHOPIFY_STOREFRONT_ACCESS_TOKEN production
```

Step 6: Or set all at once using the Vercel API:

```
vercel --prod -e DATABASE_URL="your_neon_connection_string"
```

3.3 Variable Details

DATABASE_URL

The PostgreSQL connection string for Neon. This is the ONLY variable that needs to be updated to make the full application functional. Currently set to a placeholder value. The format is:

```
postgresql://[user]:[password]@[endpoint].neon.tech/[dbname]?sslmode=require
```

Note: The DATABASE_URL must point to a real Neon PostgreSQL database. The current placeholder value will cause all database-dependent features to fail.

SHOPIFY_STORE_DOMAIN

Your Shopify store domain. This is used by both the Storefront API and Admin API to construct the base URL for API calls. Format: your-store.myshopify.com

SHOPIFY_STOREFRONT_ACCESS_TOKEN

The Storefront API access token created in your Shopify Admin under Apps > Develop apps. This token is used for storefront operations like product queries and checkout creation. Current value: 09124abbd07ca423DB9488B3DC2C737F

SHOPIFY_ADMIN_API_TOKEN

The Admin API access token (starts with shpat_). This token is used for the fallback data fetching system that keeps products and categories available even when the database is down. Current value: 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

NEXTAUTH_SECRET

A secret key used by NextAuth.js for encryption and signing. Generate one with:


```
openssl rand -base64 32
```

NEXTAUTH_URL

The canonical URL of your deployed application. This is used by NextAuth.js for callback URLs. Set this to your Vercel deployment URL: <https://my-project-sepia-seven-42.vercel.app>

4. Neon PostgreSQL Database Setup

Neon is a serverless PostgreSQL platform that provides free-tier databases perfect for Vercel deployments. This section walks you through creating a Neon database and connecting it to your Vercel project.

4.1 Creating a Neon Database

Step 1: Go to <https://console.neon.tech> and sign up (free with GitHub/Google/Email)

Step 2: Click "Create Project" or "New Project"

Step 3: Configure the project:

Setting	Recommended Value
Project Name	3boxes-luxury
Region	Asia (Mumbai / Singapore)
PostgreSQL Version	16 (latest stable)
Database Name	boxes_luxury
Branch	main (default)

Step 4: Click "Create Project" and wait for provisioning (usually 10-30 seconds)

Step 5: Once created, you will see the connection string on the dashboard

Step 6: Copy the connection string - it looks like:

```
postgresql://neondb_owner:AbCdEfGh@ep-cool-name-123456.us-east-2.aws.neon.tech/boxes_luxury?sslmode=require
```

Note: Save this connection string securely. You will need it for the DATABASE_URL environment variable on Vercel.

4.2 Neon Free Tier Limits

Resource	Free Tier Limit
----------	-----------------

Storage	0.5 GB
Compute	300 hours/month (auto-suspend)
Branches	10 branches
Projects	1 project
Rows	Unlimited (within storage)
Connections	Via connection pooling

4.3 Connecting Neon to Vercel

Once you have the Neon connection string, update the DATABASE_URL on Vercel:

Method 1: Via Vercel Dashboard

- Step 1:** Go to <https://vercel.com/dashboard>
- Step 2:** Select your project
- Step 3:** Navigate to Settings > Environment Variables
- Step 4:** Find DATABASE_URL and click Edit
- Step 5:** Replace the placeholder with your Neon connection string
- Step 6:** Save and redeploy (Deployments > Redeploy)

Method 2: Via Vercel CLI

```
# Remove old value
vercel env rm DATABASE_URL production
# Add new value
vercel env add DATABASE_URL production
# Paste your Neon connection string when prompted
# Redeploy
vercel --prod
```

Method 3: Using Neon-Vercel Integration (Recommended)

- Step 1:** Go to <https://vercel.com/integrations/neon>
- Step 2:** Click "Add Integration"
- Step 3:** Select your Vercel project
- Step 4:** This automatically creates a Neon database and sets DATABASE_URL, POSTGRES_URL, and POSTGRES_PRISMA_URL environment variables
- Step 5:** Redeploy your project

Note: The Neon-Vercel integration is the easiest method. It also sets POSTGRES_PRISMA_URL which is used by the vercel.json build command for prisma db push.

4.4 Verifying Database Connection

After updating DATABASE_URL and redeploying, verify the database connection by checking these API endpoints:

Endpoint	Method	Expected Result
/api/products	GET	Returns product list (Shopify fallback or DB)
/api/categories	GET	Returns category list
/api/cart	GET	Returns cart data (needs DB)
/api/auth/me	GET	Returns current user (needs DB)
/api/admin/stats	GET	Returns admin dashboard stats (needs DB)

5. Shopify Integration Configuration

The application integrates with Shopify through two APIs: the Admin API (for product/category data fallback) and the Storefront API (for checkout and cart functionality). This section details the complete Shopify setup.

5.1 Shopify Store Details

Property	Value
Store Domain	3boxesluxury-2.myshopify.com
Shopify Admin URL	https://3boxesluxury-2.myshopify.com/admin
API Version	2024-10
Admin API Token	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
Storefront API Token	09124abbd07ca423DB9488B3DC2C737F
Total Products	57 (active)
Total Collections	15 (custom + smart)

5.2 Creating Shopify API Credentials

If you need to regenerate or create new API credentials, follow these steps:

Step 1: Log into Shopify Admin: <https://3boxesluxury-2.myshopify.com/admin>

Step 2: Navigate to Apps > Develop apps (in the left sidebar)

Step 3: Click "Create an app"

Step 4: Name the app (e.g., "3 Boxes Luxury Headless") and click Create

Step 5: Go to Configuration > Configure Admin API Scopes

Step 6: Enable the following scopes:

Scope	Access Level	Purpose
read_products	Read	Fetch product data
read_product_listings	Read	Fetch product listings
read_custom_collections	Read	Fetch custom collections
read_smart_collections	Read	Fetch smart collections
read_inventory	Read	Check stock levels
read_orders	Read	Sync order data
read_customers	Read	Customer data sync
read_price_rules	Read	Fetch discount rules

Step 7: Click Save and then "Install app" to generate the Admin API token

Step 8: Copy the token (starts with shpat_) - it is shown only once

Step 9: For Storefront API: Go to Configuration > Storefront API

Step 10: Enable the Storefront API and select the required scopes

Step 11: Copy the Storefront API access token

5.3 Shopify API Fallback System

The application implements a fallback system in `src/lib/shopify.ts` that uses the Shopify Admin API to fetch products and categories when the PostgreSQL database is unavailable. Key features include:

- In-memory caching with 5-minute TTL ($CACHE_TTL = 5 * 60 * 1000$)
- Next.js revalidation at 300 seconds (5 minutes)
- Fetches up to 250 products per API call
- Derives categories from both Shopify collections and product types
- Search functionality filtering by name, description, and tags
- Auto-maps Shopify `product_type` to application category slugs

Note: Products fetched via the fallback have IDs prefixed with "shopify-" (e.g., shopify-123456789) to distinguish them from database-sourced products.

6. Database Schema (Prisma)

The application uses Prisma ORM with 35+ models covering e-commerce, corporate gifting, inventory management, accounting, and more. The schema is defined in `prisma/schema.prisma` and automatically pushed to the database during the Vercel build process.

6.1 Core E-Commerce Models

Model	Purpose	Key Fields
Category	Product categories	name, slug, description, image
Product	Product catalog	name, slug, price, images, stock, featured
ProductVariant	Product variants	name, sku, price, stock, attributes
ProductImage	Product images	url, alt, sort order
Cart	Shopping cart	sessionId, userId, couponCode
CartItem	Cart line items	quantity, variantId, giftWrapping
Order	Customer orders	orderNumber, total, status, trackingNumber
OrderItem	Order line items	name, price, quantity, giftWrapping
Review	Product reviews	rating, title, comment, verified
WishlistItem	User wishlist	userId, productId

6.2 User and Authentication Models

Model	Purpose	Key Fields
User	User accounts	email, name, role, adminRole, twoFactorEnabled
Session	Auth sessions	token, userId, expiresAt, deviceInfo
UserPermission	Role permissions	userId, permission
AuditLog	Activity audit	action, entity, ipAddress

6.3 Corporate Gifting Models

Model	Purpose	Key Fields
CorporateAccount	Corporate accounts	companyName, approvalStatus, creditLimit, discountPercent

CorporateMember	Team members	email, role, status
CorporateBranding	Branding config	logoUrl, primaryColor, packagingType
CorporateCampaign	Gifting campaigns	name, occasion, budget, status
CampaignRecipient	Recipients	name, email, giftStatus, orderId

6.4 Admin and Operations Models

Model	Purpose
Vendor	Product vendors and suppliers
Invoice / InvoiceItem	Vendor invoices and line items
AccountEntry	Double-entry accounting (debit/credit)
InventoryLog	Stock movement tracking
Offer	Discount codes and promotional offers
PaymentSession	Payment gateway sessions (Razorpay/Stripe)
OrderInvoice	Customer-facing order invoices
OrderTrackingEvent	Shipment tracking timeline
PlatformIntegration	External platform integrations (Myntra, Nykaa)
SyncLog	Product sync history logs
AffiliateClick	Affiliate link click tracking
SupportTicket / Message	Customer support ticket system
WikiDocument	Internal knowledge base
CurrencyRate	Multi-currency exchange rates
GeoCountry	Country detection and locale mapping

7. API Routes Reference

The application exposes 80+ API routes organized into functional groups. All routes are under /api/ and follow RESTful conventions. Authentication is handled via Bearer token in the Authorization header.

7.1 Authentication Routes

Route	Method	Description
-------	--------	-------------

/api/auth/register	POST	Register new user
/api/auth/login	POST	Login with email/password
/api/auth/logout	POST	Logout and invalidate session
/api/auth/me	GET	Get current user profile
/api/auth/session	GET	Check session validity
/api/auth/refresh	POST	Refresh auth token
/api/auth/forgot-password	POST	Request password reset
/api/auth/reset-password	POST	Reset password with token
/api/auth/otp/send	POST	Send OTP for phone login
/api/auth/otp/verify	POST	Verify OTP code
/api/auth/2fa/setup	POST	Setup two-factor auth
/api/auth/2fa/verify	POST	Verify 2FA code
/api/auth/social	POST	Social login (Google/Facebook)
/api/auth/approve	POST	Approve user (admin)
/api/auth/verify-email	POST	Verify email address
/api/auth/verify-phone	POST	Verify phone number

7.2 Product and Category Routes

Route	Method	Description
/api/products	GET	List products (supports filtering)
/api/products/[id]	GET	Get single product
/api/categories	GET	List categories
/api/search	GET	Search products by query
/api/gift-recommend	GET	AI-powered gift recommendations
/api/combo-suggestions	GET	Product combo suggestions

7.3 Shopping and Checkout Routes

Route	Method	Description
/api/cart	GET/POST	Get or update cart
/api/checkout	POST	Create checkout session

/api/checkout/estimate	POST	Estimate shipping and tax
/api/orders	GET	List user orders
/api/orders/[id]	GET	Get order details
/api/orders/[id]/tracking	GET	Get order tracking info
/api/orders/[id]/invoice	GET	Download order invoice
/api/orders/[id]/refund	POST	Request order refund
/api/payments/create-session	POST	Create payment session
/api/payments/verify	POST	Verify payment
/api/offers/validate	POST	Validate coupon code

7.4 Admin Routes

Route	Description
/api/admin/dashboard	Admin dashboard data
/api/admin/stats	Platform statistics
/api/admin/users	User management (CRUD)
/api/admin/products	Product management (CRUD)
/api/admin/categories	Category management
/api/admin/orders	Order management
/api/admin/corporate	Corporate account management
/api/admin/campaigns	Campaign management
/api/admin/coupons	Coupon management
/api/admin/permissions	Permission management
/api/admin/sessions	Active session management
/api/admin/audit-logs	Audit log viewing
/api/admin/reports	Report generation
/api/admin/role-permissions	Role-based permissions

7.5 Corporate Gifting Routes

Route	Description
/api/corporate/register	Register corporate account

/api/corporate/login	Corporate login
/api/corporate/profile	Corporate profile management
/api/corporate/members	Team member management
/api/corporate/branding	Branding configuration
/api/corporate/campaigns	Campaign CRUD
/api/corporate/campaigns/[id]/recipients	Recipient management
/api/corporate/campaigns/[id]/submit	Submit campaign for approval
/api/corporate/recipients/import-csv	Bulk import recipients via CSV

8. Complete Credentials Reference

This section contains all credentials and access details for the deployed application. Keep this document secure as it contains sensitive API keys and tokens.

8.1 Vercel Deployment

Property	Value
Project URL	https://my-project-sepia-seven-42.vercel.app
Vercel Dashboard	https://vercel.com/dashboard
Vercel Token	vcp_2mJFo9UaGarHsCeVd8wRTiwWevFJaulJdtaC9PpKwqsa1YUMZV2Dzipr
Region	bom1 (Mumbai, India)
Framework	Next.js
Build Command	Custom (see vercel.json)
Install Command	bun install

8.2 Shopify Store

Property	Value
Store Domain	3boxesluxury-2.myshopify.com
Admin URL	https://3boxesluxury-2.myshopify.com/admin
Admin API Token	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
Storefront API Token	09124abbd07ca423DB9488B3DC2C737F

API Version	2024-10
-------------	---------

8.3 Database

Property	Value
Provider (Production)	PostgreSQL (Neon Serverless)
Provider (Development)	SQLite (local file)
DATABASE_URL (Current)	postgresql://user:pass@ep-xxx.neon.tech/dbname?sslmode=require
DATABASE_URL (Required)	Real Neon connection string (needs update)
Local DB Path	file:/home/z/my-project/db/custom.db
Prisma Schema	prisma/schema.prisma
ORM	Prisma Client v6.11.1

Note: The DATABASE_URL on Vercel currently points to a placeholder. It MUST be updated with a real Neon PostgreSQL connection string for full functionality.

9. Troubleshooting Guide

9.1 Build Failures

Problem: Prisma db push fails on Vercel

- Cause: DATABASE_URL is a placeholder or invalid connection string
- Fix: Update DATABASE_URL with a valid Neon PostgreSQL connection string
- Verify: Test the connection string locally with: `npx prisma db push`

Problem: TypeScript build errors

- The project uses `ignoreBuildErrors: true` in `next.config.ts` to bypass TS errors
- This is intentional for rapid deployment; fix TS errors locally before production

Problem: SQLite to PostgreSQL switch fails

- The `vercel-build` script uses `sed` to switch the Prisma provider
- If this fails, verify that `prisma/schema.prisma` has: `provider = "sqlite"`
- The `sed` command replaces `"sqlite"` with `"postgresql"` during build

9.2 Runtime Errors

Problem: Cart/Orders/Admin pages return 500 errors

- Cause: Database connection fails (placeholder DATABASE_URL)
- Fix: Update DATABASE_URL with real Neon connection string
- Products/Categories still work via Shopify Admin API fallback

Problem: Shopify API returns 401 Unauthorized

- Cause: Invalid or expired API token
- Fix: Regenerate the token in Shopify Admin > Apps > Develop apps

Problem: Vercel Deployment Protection blocks access

- Cause: Vercel Standard/Pro plans have Deployment Protection enabled by default
- Fix: Go to Vercel Dashboard > Settings > Deployment Protection > Disable

9.3 Quick Diagnostic Commands

```
# Check Vercel deployment status
vercel ls

# View deployment logs
vercel logs https://my-project-sepia-seven-42.vercel.app

# Check environment variables
vercel env ls

# Test Shopify API connection
curl -H "X-Shopify-Access-Token: shpat_..." \
"https://3boxesluxury-2.myshopify.com/admin/api/2024-10/products/count.json"

# Test database connection locally
npx prisma db push --schema=prisma/schema.prisma

# Redeploy to production
vercel --prod
```

10. Complete Setup Checklist

Follow this checklist to get the 3 BOXES LUXURY application fully operational on Vercel:

#	Task	Status	Details
1	Create Neon PostgreSQL database	PENDING	Go to console.neon.tech, create free project
2	Copy Neon connection string	PENDING	From Neon dashboard, copy the postgresql://... URL
3	Update DATABASE_URL on Vercel	PENDING	Settings > Environment Variables > Edit DATABASE_URL

4	Set SHOPIFY_STORE_DOMAIN	DONE	3boxesluxury-2.myshopify.com
5	Set SHOPIFY_ADMIN_API_TOKEN	DONE	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
6	Set SHOPIFY_STOREFRONT_ACCESS_TOKEN	DONE	09124abbd07ca423DB9488B3DC2C737F
7	Set NEXTAUTH_SECRET	PENDING	Generate: openssl rand -base64 32
8	Set NEXTAUTH_URL	PENDING	https://my-project-sepia-seven-42.vercel.app
9	Disable Deployment Protection	PENDING	Vercel > Settings > Deployment Protection
10	Redeploy to production	PENDING	Run: vercel --prod
11	Verify products load	DONE	Products work via Shopify fallback
12	Verify cart/checkout works	PENDING	Needs real DATABASE_URL
13	Verify user registration	PENDING	Needs real DATABASE_URL
14	Verify admin dashboard	PENDING	Needs real DATABASE_URL
15	Verify corporate gifting	PENDING	Needs real DATABASE_URL

End of Document - For questions or support, refer to the Vercel documentation at <https://vercel.com/docs> and the Neon documentation at <https://neon.tech/docs>.